

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №32

Тема: Створення алгоритмів на опрацювання тексту

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати основні алгоритми опрацювання текстових даних: розбиття тексту на слова, підрахунок частот за допомогою словників, перевірку паліндромів і анаграм, шифри заміни (Цезаря), стиснення повторів (RLE), порівняння текстів; закріпити роботу з рядками, індексами, зрізами та методами str.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Рядок як послідовність символів. У Python рядок (str) — незмінна послідовність символів Unicode. Доступні індексування `s[i]` (від 0; `s[-1]` — останній символ), зрізи `s[a:b:k]`, довжина `len(s)`, конкатенація `+`, повторення `*` та перебір `for ch in s`. Оскільки рядок незмінний, результат кожної зміни — **новий** рядок; для накопичення результату в циклі ефективніше збирати частини в список і об'єднувати `"".join(parts)`.

2. Корисні методи str:

Метод	Призначення	Приклад
<code>lower()</code> , <code>upper()</code>	перехід до нижнього / верхнього регістру	<code>"Київ".lower() → "київ"</code>
<code>split()</code> , <code>split(sep)</code>	розбиття на список підрядків	<code>"a б в".split() → ["a", "б", "в"]</code>
<code>join(list)</code>	об'єднання списку рядків	<code>"-".join(["a", "б"]) → "a-б"</code>
<code>strip()</code>	видалення пробілів по краях	<code>" ок ".strip() → "ок"</code>
<code>find(sub)</code> , <code>count(sub)</code>	перша позиція (–1, якщо немає) / кількість входжень	<code>"abab".count("ab") → 2</code>
<code>replace(a, b)</code>	заміна підрядків	<code>"aab".replace("a", "x") → "xxb"</code>
<code>isalpha()</code> , <code>isdigit()</code>	перевірка типу символів	<code>"a1".isalpha() → False</code>
<code>ord(ch)</code> , <code>chr(n)</code>	код Unicode символу / символ за кодом	<code>ord("a") → 97</code>

3. Словник для підрахунку частот. Для підрахунку, скільки разів трапляється кожне слово чи символ, використовують словник: `freq[w] = freq.get(w, 0) + 1`, або спеціалізований клас `collections.Counter`, метод `most_common(k)` якого повертає `k` найчастіших елементів. Складність підрахунку $O(n)$. Для впорядкування результату за частотою застосовують `sorted(freq.items(), key=lambda p: (-p[1], p[0]))` (спочатку за спаданням частоти, при рівних — за алфавітом).

4. Нормалізація тексту. Перед порівнянням текст приводять до єдиного вигляду: нижній регістр, вилучення розділових знаків і пробілів (залишають лише літери та цифри за умовою `ch.isalnum()`). Саме так перевіряють **паліндроми** (рядок, що однаково читається в обох напрямках: «А роза упала на лапу Азора») та **анаграми** (слова з однаковим набором літер: «кіт» — «тік» так, а «мама» — «мимо» ні): два слова є анаграмами, якщо рівні відсортовані послідовності їхніх літер або їхні словники частот.

5. Шифр Цезаря. Кожну літеру алфавіту (з N літер) замінюють літерою, зсунутою на k позицій: $c' = (c + k) \bmod N$. Розшифрування – зсув на $-k$. Для англійського алфавіту $N = 26$, для українського $N = 33$. Шифр легко зламати перебором усіх $N - 1$ ключів або частотним аналізом, тому має лише навчальне значення.

6. Стиснення повторів (RLE, run-length encoding). Послідовність однакових символів замінюють парою «символ + кількість»: `aaabccddddd` → `a3b1c2d4`. Метод ефективний для даних із довгими серіями повторів (прості зображення), але для звичайного тексту може збільшити розмір.

7. Відстань між текстами. Найпростіша міра подібності — частка збігів на однакових позиціях (відстань Геммінга для рядків однакової довжини: кількість позицій, на яких символи різні). Загальніша міра — відстань Левенштейна (практична робота №29).

Хід роботи

Завдання 1. Ручна обробка тексту: підрахунок частот і паліндроми

Дано текст: «Мама мила раму. Мама мила Мілу.» Виконати його обробку вручну.

Порядок виконання:

1. Приведіть текст до нижнього регістру й вилучіть розділові знаки; запишіть список слів і їх кількість.
2. Складіть таблицю частот слів «слово — частота» і впорядкуйте її за спаданням частоти (при рівних частотах — за алфавітом).
3. Підрахуйте кількість літер у тексті (без пробілів і розділових знаків) і складіть таблицю частот перших п'яти літер.
4. Перевірте, чи є паліндромами слова «мама», «потоп», «шалаш» та фраза «А роза упала на лапу Азора»: для фрази спочатку нормалізуйте її (нижній регістр, без пробілів) і порівняйте з рядком у зворотному порядку.
5. Перевірте, чи є анаграмами пари слів «кіт» — «тік» та «мама» — «мимо»: запишіть відсортовані послідовності літер.

Завдання 2. Ручне шифрування та стиснення

Виконати вручну шифрування шифром Цезаря та стиснення методом RLE.

Порядок виконання:

1. Зашифруйте слово «CAESAR» шифром Цезаря з ключем $k = 3$ (англійський алфавіт із 26 літер): для кожної літери запишіть її номер ($A = 0$), номер після зсуву за формулою $(c + k) \bmod 26$ та отриману літеру.
2. Розшифруйте слово «KHOOR» (ключ 3) і поясніть, як змінюється формула при розшифруванні.
3. Зашифруйте слово «XYZ» ключем 5 і поясніть, навіщо потрібна операція $\bmod 26$ (перехід через кінець алфавіту).

4. Стисніть методом RLE рядок «aaabscddddd» і рядок «abcd»; обчисліть довжину початкового й стисненого рядків та коефіцієнт стиснення в кожному випадку.
5. Розпакуйте рядок «x5y2z1» і зробіть висновок про те, на яких даних RLE доцільно застосовувати.

Завдання 3. Програмна реалізація: частоти, паліндроми, анаграми

Реалізувати мовою Python алгоритми обробки тексту та перевірити їх на прикладах із ручних розрахунків.

Порядок виконання:

1. Напишіть функцію `words(text)`, що повертає список слів тексту в нижньому регістрі без розділових знаків (пройдіть текст циклом і накопичуйте літери; слово завершується будь-яким нелітерним символом).
2. Напишіть функцію `word_freq(text)`, що повертає список пар (слово, частота), впорядкований за спаданням частоти й за алфавітом при рівних частотах (спочатку реалізуйте через звичайний словник, потім перевірте через `Counter`).
3. Напишіть функцію `is_palindrome(text)` із нормалізацією (залишайте лише літери та цифри в нижньому регістрі) та функцію `is_anagram(a, b)` через порівняння словників частот літер.
4. Напишіть функцію `longest_word(text)`, що повертає найдовше слово (при рівній довжині — перше за порядком), та функцію `vowels_count(text)`, що підраховує голосні літери українського алфавіту «аєєіїоуоя».
5. Перевірте функції на даних завдання 1 та на тестах із порожнім рядком, рядком із самих пробілів, словом з однієї літери.

Приклад реалізації мовою Python:

python

```
from collections import Counter

def words(text):
    result, cur = [], ""
    for ch in text:
        if ch.isalpha() or ch == "'":
            cur += ch.lower()
        elif cur:
            result.append(cur)
            cur = ""
    if cur:
        result.append(cur)
    return result

def word_freq(text):
    freq = {}
    for w in words(text):
        freq[w] = freq.get(w, 0) + 1
    return sorted(freq.items(), key=lambda p: (-p[1], p[0]))

def is_palindrome(text):
    s = "".join(ch.lower() for ch in text if ch.isalnum())
    return s == s[::-1]

def is_anagram(a, b):
    na = "".join(ch.lower() for ch in a if ch.isalpha())
    nb = "".join(ch.lower() for ch in b if ch.isalpha())
    return Counter(na) == Counter(nb)
```

```

def longest_word(text):
    best = ""
    for w in words(text):
        if len(w) > len(best):
            best = w
    return best

def vowels_count(text):
    return sum(1 for ch in text.lower() if ch in "аеєіїоуя")

text = "Мама мила раму. Мама мила Мілу."
print("Слова:", words(text))
print("Частоти:", word_freq(text))
print("Збіг із Counter:", word_freq(text) == sorted(Counter(words(text)).items(),
key=lambda p: (-p[1], p[0])))
print("Найдовше слово:", longest_word(text), "; голосних:", vowels_count(text))
for s in ("мама", "потоп", "шалаш", "А роза упала на лапу Азора", "програма"):
    print(f"Паліндром {s!r}: {is_palindrome(s)}")
print("Анаграми кіт/тік:", is_anagram("кіт", "тік"), "; мама/мимо:", is_anagram("мама",
"мимо"))
print("Порожній рядок:", words(""), word_freq(""), is_palindrome(""))
print("Слово з однієї літери:", words("я"), is_palindrome("я"))

```

Результат виконання програми:

```

Слова: ['мама', 'мила', 'раму', 'мама', 'мила', 'мілу']
Частоти: [('мама', 2), ('мила', 2), ('мілу', 1), ('раму', 1)]
Збіг із Counter: True
Найдовше слово: мама ; голосних: 12
Паліндром 'мама': False
Паліндром 'потоп': True
Паліндром 'шалаш': True
Паліндром 'А роза упала на лапу Азора': True
Паліндром 'програма': False
Анаграми кіт/тік: True ; мама/мимо: False
Порожній рядок: [] [] True
Слово з однієї літери: ['я'] True

```

Висновок за результатами: текст містить 6 слів, із яких «мама» та «мила» повторюються по 2 рази — це збігається з ручною таблицею частот. Фраза «А роза упала на лапу Азора» є паліндромом після нормалізації (пробіли й регістр не враховуються), а слово «програма» — ні. Анаграми «кіт» і «тік» визначаються порівнянням словників частот літер, а для порожнього рядка всі функції повертають коректний порожній результат без помилок — обробку граничних випадків необхідно перевіряти окремо.

Завдання 4. Шифр Цезаря, злам шифру та стиснення RLE

Реалізувати шифр Цезаря для англійського й українського алфавітів, злам шифру перебором і за частотним аналізом, а також стиснення та розпакування RLE.

Порядок виконання:

1. Напишіть функцію `caesar(text, k, alphabet)`, що зсуває лише літери, які належать алфавіту (великі та малі окремо, решта символів залишається без змін); розшифрування виконується викликом із ключем `-k`.
2. Зашифруйте та розшифруйте фразу «Hello, World!» ($k = 3$) та «Привіт, Україно!» ($k = 5$, алфавіт із 33 літер); переконайтеся, що після розшифрування отримується початковий текст.

3. Напишіть функцію `crack_brute(cipher, alphabet)`, що виводить усі варіанти розшифрування для $k = 1 \dots N-1$; знайдіть вручну осмислений варіант для шифртексту «Wklv lv d whvw».
4. Реалізуйте частотний злам: припустіть, що найчастіша літера шифртексту — це «е» (для англійського тексту), визначте ключ і розшифруйте довший текст; перевірте результат.
5. Напишіть функції `rle_encode(s)` та `rle_decode(s)` і перевірте їх на прикладах із завдання 2, а також на випадкових рядках (розпакування після стиснення має відновлювати рядок). Знайдіть рядок, для якого RLE збільшує довжину.

Приклад реалізації мовою Python:

python

```
from collections import Counter
import itertools

EN = "abcdefghijklmnopqrstuvwxyz"
UA = "абвгдеєжзиіїйклмнопрстуфхцщъя"

def caesar(text, k, alphabet=EN):
    n = len(alphabet)
    out = []
    for ch in text:
        low = ch.lower()
        if low in alphabet:
            new = alphabet[(alphabet.index(low) + k) % n]
            out.append(new.upper() if ch.isupper() else new)
        else:
            out.append(ch)
    return "".join(out)

def crack_brute(cipher, alphabet=EN):
    return [(k, caesar(cipher, -k, alphabet)) for k in range(1, len(alphabet))]

def crack_frequency(cipher, alphabet=EN, common="e"):
    letters = [ch.lower() for ch in cipher if ch.lower() in alphabet]
    top = Counter(letters).most_common(1)[0][0]
    k = (alphabet.index(top) - alphabet.index(common)) % len(alphabet)
    return k, caesar(cipher, -k, alphabet)

def rle_encode(s):
    return "".join(ch + str(len(list(group))) for ch, group in itertools.groupby(s))

def rle_decode(s):
    result, i = [], 0
    while i < len(s):
        ch, j = s[i], i + 1
        while j < len(s) and s[j].isdigit():
            j += 1
        result.append(ch * int(s[i + 1:j]))
        i = j
    return "".join(result)

enc = caesar("Hello, World!", 3)
print(enc, "->", caesar(enc, -3))
enc = caesar("Привіт, Україно!", 5, UA)
print(enc, "->", caesar(enc, -5, UA))
print("Brute force для 'Wklv lv d whvw':")
```

```

for k, plain in crack_brute("Wklv lv d whvw")[:4]:
    print(f"  k = {k}: {plain}")

secret = caesar("Meet me near the green tree. The new secret key is better than the
evening reference.", 7)
print("Шифртекст:", secret)
print("Частотний злам:", crack_frequency(secret))

for s in ("aaabccddddd", "abcd", "x" * 12):
    packed = rle_encode(s)
    print(f"{s!r:>14} -> {packed!r:<10} довжина {len(s)} -> {len(packed)}; розпаковано
правильно: {rle_decode(packed) == s}")
print("RLE для 'abcd' збільшує довжину у", len(rle_encode("abcd")) / len("abcd"),
"раза")

```

Результат виконання програми:

```

Khoor, Zruog! -> Hello, World!
Фхлемч, Шпхднту! -> Привіт, Україно!
Brute force для 'Wklv lv d whvw':
  k = 1: Vjku ku c vguv
  k = 2: Uijt jt b uftu
  k = 3: This is a test
  k = 4: Sghr hr z sdrs
Шифртекст: Tlla tl ulhy aol nyllu ayll. Aol uld zljyla rlf pz ilaaly aohu aol lclupun
ylmlylujl.
Частотний злам: (7, 'Meet me near the green tree. The new secret key is better than the
evening reference.')
  'aaabccddddd' -> 'a3blc2d4' довжина 10 -> 8; розпаковано правильно: True
  'abcd' -> 'alblclld1' довжина 4 -> 8; розпаковано правильно: True
  'xxxxxxxxxxxx' -> 'x12' довжина 12 -> 3; розпаковано правильно: True
RLE для 'abcd' збільшує довжину у 2.0 раза

```

Висновок за результатами: шифр Цезаря коректно обробляє великі й малі літери та залишає без змін розділові знаки; розшифрування ключем $-k$ відновлює початковий текст. Злам перебором має лише 25 варіантів, і серед них легко знайти осмислений («This is a test» при $k = 3$), а частотний аналіз знаходить ключ автоматично (тут ключ дорівнює 7), якщо текст достатньо довгий. RLE дає вигоду для рядка з довгих серій («x» $\times$ 12 стискається до «x12»), але подвоює довжину рядка «abcd», тому застосовувати його слід лише для даних із довгими повторами.

Контрольні питання

1. Чому рядки в Python називають незмінними? Як ефективно збирати результат у циклі?
2. Які методи рядків ви використовували для розбиття, об'єднання, пошуку та заміни?
3. Як за допомогою словника підрахувати частоти слів? Як впорядкувати результат за частотою, а при рівних частотах — за алфавітом?
4. Що таке нормалізація тексту і навіщо вона потрібна при перевірці паліндромів та анаграм?
5. Як працює шифр Цезаря? Чому його легко зламати?
6. У чому суть частотного аналізу шифртексту?
7. Опишіть алгоритм RLE. Для яких даних він ефективний, а для яких — шкідливий?
8. Що показує відстань Геммінга і чим вона відрізняється від відстані Левенштейна?

Зміст звіту

1. Тема, мета роботи.
2. Ручні таблиці частот, шифрування й стиснення (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Висновки.